

CHAPTER 6: TWO-DIMENSIONAL VIEWING

Exam Notes — Simple & Structured

Basic Idea (Introduction)

Imagine you are **taking a photo with a camera**.

- The **real-world scene** contains many objects. These are in **World Coordinates**.
- You choose **which part of the scene you want to capture**. This selected area is called the **Window**.
- The captured image is then **displayed on the screen**. The area on the screen where it appears is called the **Viewport**.

In simple words:

- **Window = What** part of the scene to show (in **World Coordinates**).
- **Viewport = Where** to show that selected part on the **display screen** (in **Device Coordinates**)

Viewing Transformation is the process of converting the selected part of a scene (Window) from World Coordinates into a Viewport on the display screen (Device Coordinates).

1. The Viewing Pipeline

Key Definitions

Term	Meaning
Window	A world-coordinate area selected for display. Defines WHAT is to be viewed.
Viewport	An area on the display device where the window is mapped. Defines WHERE it is displayed.
Viewing Transformation	Mapping a part of a world-coordinate scene to device coordinates.
Window-to-Viewport Transformation	Simple name for the 2D viewing transformation (also called windowing transformation).

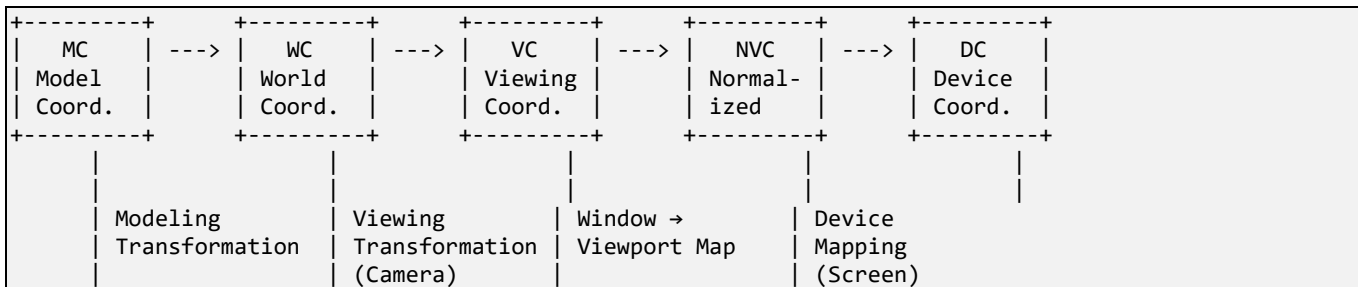
Note

- Usually, both the Window and the Viewport are rectangular and their sides are parallel to the X-axis and Y-axis. This makes calculations simple and fast.
- Other shapes, such as circles or polygons, can also be used as windows, but they require more complex calculations and take more processing time.
- The Window in computer graphics is not the same as a GUI (Graphical User Interface) window. Here, a Window means the selected area of the picture or scene that we want to display, not a movable application window on the screen.

1. Model Coordinates (MC): Model Coordinates (MC) are the coordinates used to create an object in its own coordinate system before it is placed in the scene. **Example:** A car is designed with its center at **(0,0)**.

2. **World Coordinates (WC):** World Coordinates (WC) are the coordinates that show the position of an object in the whole scene or world. **Example:** The car is placed at **(250,120)** in the world.
3. **Viewing Coordinates (VC):** Viewing Coordinates (VC) are the coordinates of objects as seen from the viewer or camera. **Example:** The camera shows only the objects in front of it.
4. **Normalized Viewing Coordinates (NVC):** Normalized Viewing Coordinates (NVC) are coordinates converted into a standard range from **0 to 1** so they can be displayed on any screen.
5. **Device Coordinates (DC):** Device Coordinates (DC) are the final screen coordinates used to display objects in pixels on the monitor. **Example:** A point is displayed at pixel **(500,300)**.

The Viewing Transformation Pipeline



Steps of the Viewing Transformation Pipeline

Step 1: Modeling Transformation

Convert Model Coordinates (MC) into World Coordinates (WC) by placing the object in the scene.

Step 2: Viewing Transformation

Convert World Coordinates (WC) into Viewing Coordinates (VC) using the viewing reference frame.

Step 3: Window-to-Viewport Mapping

Convert Viewing Coordinates (VC) into Normalized Viewing Coordinates (NVC).

Step 4: Clipping

Remove all objects or object parts that lie outside the viewing window.

Step 5: Device Mapping

Convert Normalized Viewing Coordinates (NVC) into Device Coordinates (DC) for display on the monitor.

Why use Normalized Coordinates (0 to 1 range)?

Normalized Viewing Coordinates (0–1 range) are used because:

- They make the graphics system **device-independent**.
- The same scene can be displayed on different screen sizes.
- Only the final mapping to the device changes.
- It simplifies the graphics pipeline.

Zooming & Panning (easy trick to remember)

- Zoom IN → make the window smaller (view fine details).
- Zoom OUT → make the window larger.
- Pan → move a fixed-size window across the scene.
- Changing viewport size/position → changes size/position of the object on the screen.

2. Viewing Coordinate Reference Frame

The Viewing Coordinate Reference Frame (VRC) is a coordinate system attached to the viewer (camera). It defines the camera's position and orientation, allowing the viewing window to be translated or rotated before transforming objects from world coordinates to viewing coordinates.

Steps to Set Up Viewing Coordinates

- Choose a viewing origin: point $P_o = (x_o, y_o)$ in world coordinates. This point represents the position of the viewer (camera).
- Choose a View-Up Vector V — this tells the direction of the viewing y-axis (y_v).
- From V , calculate two unit vectors:
 - $v = (v_x, v_y) \rightarrow$ direction of viewing y_v axis
 - $u = (u_x, u_y) \rightarrow$ direction of viewing x_v axis (perpendicular to v)
- These vectors u and v form the rows of a Rotation Matrix R that aligns viewing axes with world axes.

Composite Transformation Formula (World $\rightarrow$ Viewing Coordinates)

$$M_{WC, VC} = R \cdot T$$

Where:

- T = Translation matrix $\rightarrow$ moves viewing origin P_o to world origin $(0,0)$
- R = Rotation matrix $\rightarrow$ aligns viewing axes (x_v, y_v) with world axes (x_u, y_u)

3. Window-to-Viewport Coordinate Transformation

Window-to-Viewport Coordinate Transformation is the process of converting the coordinates of an object from the Window (viewing area) to the Viewport (display area on the screen) while keeping the object's relative position and shape unchanged.

Purpose

The main purpose is to display the selected part of the scene (Window) inside the Viewport on the screen. It ensures that the relative position of every point remains the same.

Example: If a point is at the center of the Window, it will also appear at the center of the Viewport.

Point Mapping

Suppose,

- (x_w, y_w) = Coordinates of a point in the **Window**
- (x_v, y_v) = Coordinates of the same point in the **Viewport**

The mapping is given by

$$\frac{x_v - x_{vmin}}{x_{vmax} - x_{vmin}} = \frac{x_w - x_{wmin}}{x_{wmax} - x_{wmin}}$$
$$\frac{y_v - y_{vmin}}{y_{vmax} - y_{vmin}} = \frac{y_w - y_{wmin}}{y_{wmax} - y_{wmin}}$$

These equations ensure that the **relative position** of the point remains unchanged.

Window-to-Viewport Transformation Equations

The viewport coordinates are calculated as

$$\begin{aligned}x_v &= x_{vmin} + (x_w - x_{wmin}) \times s_x \\ y_v &= y_{vmin} + (y_w - y_{wmin}) \times s_y\end{aligned}$$

where

- s_x = Scaling factor in the x-direction
- s_y = Scaling factor in the y-direction

Scaling Factors

$$\begin{aligned}s_x &= \frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}} \\ s_y &= \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}}\end{aligned}$$

These scaling factors determine how much the window is enlarged or reduced to fit inside the viewport.

Alternate Way to Derive (2-step transformation)

- Scale the window to the size of the viewport (using fixed point = $(x_u \text{ min}, y_u \text{ min})$).
- Translate the scaled window to the position of the viewport.

Aspect Ratio: The **Aspect Ratio** is the ratio between the width and height of an object.

Case 1: $s_x = s_y$

- The object's shape is **preserved**.
- **No distortion** occurs.

Case 2: $s_x \neq s_y$

- The object becomes **stretched** or **compressed**.
- **Distortion** occurs.

4. Two-Dimensional Viewing Functions (PHIGS Functions)

These are function calls (as used in PHIGS graphics package) to implement viewing.

(a) Set viewing coordinate system

```
evaluateViewOrientationMatrix(x0, y0, xV, yV, error, viewMatrix)
```

- $x0, y0 \rightarrow$ viewing origin
- $xV, yV \rightarrow$ view-up vector direction
- Produces viewMatrix (the world-to-viewing matrix)

(b) Set window-to-viewport mapping

```
evaluateViewMappingMatrix(xwmin, xwmax, ywmin, ywmax,  
                          xvmin, xvmax, yvmin, yvmax,  
                          error, viewMappingMatrix)
```

- First 4 params $\rightarrow$ window limits (viewing coordinates)
- Next 4 params $\rightarrow$ viewport limits (normalized coordinates)

(c) Store combination in a Viewing Table

```
setViewRepresentation(ws, viewIndex, viewMatrix, viewMappingMatrix,  
                    xclipmin, xclipmax, yclipmin, yclipmax, clipxy)
```

- ws → workstation (output device)
- viewIndex → ID number for this window-viewport pair
- clipxy → set to clip or noclip
 - clip → clipping is ON
 - noclip → clipping OFF (used to view parts outside viewport too, or to speed up processing if scene is already fully inside)

(d) Select the view to use

```
setViewIndex(viewIndex)
```

This applies the chosen view settings to all following output primitives.

(e) Workstation Window & Viewport (final device mapping)

```
setWorkstationWindow(ws, xwsWindmin, xwsWindmax, ywsWindmin, ywsWindmax)  
setWorkstationViewport(ws, xwsVPortmin, xwsVPortmax, ywsVPortmin, ywsVPortmax)
```

- Workstation window: given in normalized coordinates (0 to 1)
- Workstation viewport: given in integer device coordinates

Example 6-1: Two-Dimensional Viewing Example

Sets up a window rotated 45° clockwise (view-up vector = (1,1)), origin at world origin, mapped to upper-right corner of workstation 2, with view index = 5.

```
evaluateViewOrientationMatrix(0, 0, 1, 1, viewError, viewMat);  
evaluateViewMappingMatrix(-60.5, 41.24, -20.75, 82.5,  
                          0.5, 0.8, 0.7, 1.0, viewMapError, viewMapMat);  
setViewRepresentation(2, 5, viewMat, viewMapMat, 0.5, 0.8, 0.7, 1.0, clip);  
setViewIndex(5);
```

Two views can be used together to display two graphs at opposite corners of the screen (view index 5 = upper right, view index 6 = lower left).

5. Clipping Operations

Definition: *Clipping* is the process of determining which parts of a graphical object lie **inside** a specified region and which lie **outside** it.

Applications of Clipping

- Extracting part of a scene for viewing
- Identifying visible surfaces (3D)
- Anti-aliasing line/object boundaries
- Solid-modeling procedures
- Multi-window display systems
- Drawing/painting operations (copy, move, erase, duplicate parts)

Where Can Clipping Be Done?

Method	Description
World-coordinate clipping	Clip first in world coordinates, before transforming to device coords. Saves processing — objects outside are removed early.
Viewport clipping	Transform everything to device/normalized coordinates first, then clip against viewport. Allows concatenation of transformation matrices (fewer matrix computations) but wastes effort transforming things that get discarded anyway.

Types of Clipping (Primitives)

- Point Clipping
- Line Clipping (straight line segments)
- Area/Polygon Clipping
- Curve Clipping
- Text Clipping

6. Point Clipping

Point Clipping is the process of checking whether a point lies **inside** or **outside** the clipping window (or viewport).

Rule

A point **P = (x, y)** is displayed only if:

$$\begin{aligned}x_{w \min} &\leq x \leq x_{w \max} \\ y_{w \min} &\leq y \leq y_{w \max}\end{aligned}$$

These conditions are known as the Point Clipping Test

Where:

- $x_{w \min}$ = Left boundary of the clipping window
- $x_{w \max}$ = Right boundary
- $y_{w \min}$ = Bottom boundary
- $y_{w \max}$ = Top boundary

7. Line Clipping

Basic Idea

Line Clipping is the process of removing the portions of a line that lie outside the clipping window while keeping the portion inside the window.

- Test if the line lies completely inside the window → Trivial Accept (keep it fully).
- Test if the line lies completely outside the window → Trivial Reject (discard it fully).
- If neither → the line crosses a boundary → calculate the intersection point(s) and clip.

Parametric Line Representation

Instead of treating a line as two separate equations, line clipping uses a parametric equation.

For endpoints:

$$P_1 = (x_1, y_1)$$
$$P_2 = (x_2, y_2)$$

The line is represented as:

$$x = x_1 + u(x_2 - x_1)$$
$$y = y_1 + u(y_2 - y_1)$$

where

$$0 \leq u \leq 1$$

This is called the Parametric Line Equation

Example

Given:

$$P_1 = (2, 3)$$
$$P_2 = (10, 7)$$

The parametric equations are:

$$x = 2 + 8u$$
$$y = 3 + 4u$$

Suppose an intersection occurs at

$$u = 0.5$$

Then

$$x = 2 + 8(0.5) = 6$$
$$y = 3 + 4(0.5) = 5$$

So the intersection point is:

$$(6, 5)$$

Since $0 \leq 0.5 \leq 1$, the point lies on the line segment and is a valid clipping point.

8. Cohen–Sutherland Line Clipping

Definition

The Cohen–Sutherland Line Clipping Algorithm is one of the oldest and most widely used line-clipping algorithms. It uses 4-bit region codes (outcodes) to quickly determine whether a line is:

- Completely inside (**Trivial Accept**)
- Completely outside (**Trivial Reject**)
- Partially inside (**Needs Clipping**)

This reduces the number of intersection calculations, making the algorithm efficient.

Basic Concept: Instead of immediately calculating intersection points, the algorithm first assigns a 4-bit binary region code to each endpoint. Using these codes, it performs quick logical tests to decide whether clipping is necessary.

Region Code (4-bit binary code)

Every endpoint gets a 4-digit binary code based on its position relative to the clip window.

Bit positions (from right to left):

Bit	Meaning
Bit 1	Left of window
Bit 2	Right of window
Bit 3	Below window
Bit 4	Above window

- Bit = 1 → point is in that position
- Bit = 0 → point is NOT in that position
- Code = 0000 → point is INSIDE the window

The 9 Regions (like a Tic-Tac-Toe grid around window)(unnecessary)

1001	1000	1010
0001	0000	0010
	(Window)	
0101	0100	0110

How to Compute the Region Code

For a point (x, y) :

- **Left bit = 1** if

$$x < x_{w \min}$$

- **Right bit = 1** if

$$x > x_{w \max}$$

- **Bottom bit = 1** if

$$y < y_{w \min}$$

- **Top bit = 1** if

$$y > y_{w \max}$$

Otherwise, the corresponding bit is **0**.

The Three Main Tests

Test	Condition	Result
Trivial Accept	Both endpoints have region code 0000	Line is completely INSIDE → keep entire line
Trivial Reject	Both codes have a 1 in the same bit position (check using logical AND; if result ≠ 0000)	Line is completely OUTSIDE → discard entire line
Needs Clipping	Neither of the above	Must calculate intersection with boundary

Trick: Perform logical AND of both region codes. If result = 0000 → cannot trivially reject (may need clipping). If result ≠ 0000 → trivially reject (line is completely outside).

Clipping Order (standard)

Check boundaries in this order: Left → Right → Bottom → Top

Intersection Point Formulas

1. Vertical Boundary (Left or Right edge – x is known)

$$y = y_1 + m(x - x_1)$$

where $x = x_{w_{min}}$ or $x = x_{w_{max}}$.

2. Horizontal Boundary (Bottom or Top edge – y is known)

$$x = x_1 + \frac{(y - y_1)}{m}$$

where $y = y_{w_{min}}$ or $y = y_{w_{max}}$.

Slope Formula

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

These formulas are used in **line clipping** to calculate the intersection point of a line with the clipping window boundaries.

Step-by-Step Working Example (Concept)

- Take an endpoint that is outside the window.
- Check which boundary it violates (using its region code).
- Find the intersection point with that boundary; discard the outside portion.
- Replace the outside endpoint with this new intersection point.
- Repeat with the other endpoint / remaining boundaries until the line is either fully accepted or fully rejected.

9. Polygon Clipping

Polygon Clipping is the process of removing the portions of a polygon that lie outside the clipping window while preserving the portion inside as a closed polygon. Unlike line clipping, the output must remain a complete, connected polygon so it can be filled or rendered correctly.

Why Can't We Use Line Clipping for Polygons?

A polygon is made up of several connected edges.

If we clip each edge independently using a line-clipping algorithm, we may get:

- Disconnected line segments
- Missing edges
- An open boundary instead of a closed polygon

As a result, the polygon cannot be filled (scan-converted) correctly.

Requirement of Polygon Clipping

The output must be:

- A new polygon
- With connected edges
- Forming a closed boundary
- Ready for filling (scan conversion)

10. Sutherland–Hodgman Polygon Clipping

Basic Idea

The Sutherland–Hodgman Polygon Clipping algorithm clips a polygon by processing one clipping window edge at a time (Left → Right → Bottom → Top). After clipping against one edge, the resulting polygon is used as the input for the next edge until all four boundaries are processed.

Process

- Take all polygon vertices and clip against the Left boundary → get a new vertex list.
- Pass this new list to clip against the Right boundary → newer list.
- Pass to Bottom boundary clipper → newer list.
- Pass to Top boundary clipper → Final clipped polygon.

Input Vertices → [Left Clipper] → [Right Clipper] → [Bottom Clipper] → [Top Clipper] → Output

The 4 Cases (for each pair of adjacent vertices, checked against ONE boundary at a time)

Case	First Vertex	Second Vertex	Action (added to output list)
1	Outside	Inside	Add intersection point + second vertex
2	Inside	Inside	Add only the second vertex
3	Inside	Outside	Add only the intersection point
4	Outside	Outside	Add nothing

Memory trick: Out → In: save the crossing point AND the new point. In → In: just save the new point. In → Out: save only the crossing point. Out → Out: save nothing.

Limitation of Sutherland–Hodgman Algorithm

- Works perfectly for convex polygons.
- For concave polygons, it may add an extra unwanted edge, because the algorithm always joins the last vertex to the first vertex in the output list — even if the polygon should have split into 2+ separate pieces.

Fixes for Concave Polygons

- Split the concave polygon into 2 or more convex polygons first (e.g., using the vector method — cross product of edge vectors; if any z-component is negative → concave, split there).
- Modify Sutherland-Hodgman to check for multiple vertex points along a boundary and join them correctly.
- Use a more general clipper: Weiler–Atherton algorithm or the Weiler algorithm (handles concave polygons correctly by sometimes following the polygon boundary and sometimes following the window boundary).

11. Curve Clipping

Curve Clipping is the process of removing the portions of curved objects (such as circles and ellipses) that lie outside the clipping window while displaying only the visible parts inside.

Why Different from Line/Polygon Clipping?

Curved objects (circles, ellipses) involve non-linear (curve) equations, so clipping requires more computation than straight-line boundaries.

Step-by-Step Approach

- Bounding Rectangle Test (first, quick check): Find the smallest rectangle that completely contains the curved object.
 - If bounding rectangle is completely inside → save whole object (no further computation needed).
 - If bounding rectangle is completely outside → discard whole object.
 - If it overlaps partially → go to next step.
- Further Optimization (before full curve-line intersection):
 - For a circle: Check the coordinate extents of individual quadrants. If necessary, further divide into octants for early acceptance or rejection.
 - For an ellipse: check coordinate extents of individual quadrants.
- Final Step: Solve the simultaneous line-curve equations to find actual clipping/intersection points (only done when necessary).

Same bounding-box-first approach also works when clipping curves against a general polygon clip region.

12. Text Clipping

Text Clipping is the process of displaying only the portion of text that lies inside the clipping window. There are 3 methods, ranging from fastest/crudest to slowest/most precise.

Method 1: All-or-None String Clipping (Fastest)

- Consider a bounding rectangle around the entire text string.
- If the whole string's bounding box is inside the clip window → keep the whole string.
- If any part is outside → discard the entire string.
- Fastest method, but crude (loses whole string even for minor overlap).

Method 2: All-or-None Character Clipping

- Consider a bounding rectangle around each individual character.
- If a character is completely inside → keep it.
- If a character overlaps or is outside the boundary → discard that character only (not the whole string).
- More precise than Method 1, but still doesn't clip partial characters.

Method 3: Individual Character Clipping (Most Precise)

- Each character is clipped separately instead of clipping the whole text.
- If only part of a character is outside the clipping window, only that outside part is removed.
- Outline fonts: The character is made of lines, so line-clipping is used.

- Bitmap fonts: The character is made of pixels, so each pixel is checked and clipped if it lies outside the clipping window.
- This method is the most accurate, but also the slowest.

Summary Table

Method	Unit of Clipping	Speed	Precision
All-or-none string	Whole string	Fastest	Lowest
All-or-none character	Each character	Medium	Medium
Individual character components	Parts of a character (lines/pixels)	Slowest	Highest